

Dynamic Memory Allocation in C (Review)

Dynamic memory allocation means **allocating memory at run time**. In C, dynamic memory is allocated from **heap memory** using standard library functions.

Header file required:

```
#include <stdlib.h>
```

1. `malloc()` — Memory Allocation

Purpose:

Allocates a single block of memory of specified size (in bytes).

Syntax:

```
data_type *ptr;  
ptr = (data_type *)malloc(size_in_bytes);
```

Using `sizeof()` (recommended):

```
ptr = (data_type *)malloc(n * sizeof(data_type));
```

Key Points: - Allocates **contiguous memory** - Memory is **uninitialized** (contains garbage values) - Returns `NULL` if allocation fails - Typecasting is optional in C

Example:

```
int *ptr = (int *)malloc(5 * sizeof(int));
```

2. `calloc()` — Contiguous Allocation

Purpose:

Allocates memory for multiple blocks and initializes all bytes to **zero**.

Syntax:

```
data_type *ptr;  
ptr = (data_type *)calloc(n, sizeof(data_type));
```

Where: - `n` = number of blocks - `sizeof(data_type)` = size of each block

Key Points: - Allocates **contiguous memory** - All elements are initialized to **zero** - Safer than `malloc()` when initialization is required - Returns `NULL` if allocation fails

Example:

```
int *ptr = (int *)calloc(5, sizeof(int));
```

3. `realloc()` — Reallocation

Purpose:

Changes the size of previously allocated memory.

Syntax:

```
new_ptr = (data_type *)realloc(old_ptr, new_size_in_bytes);
```

Key Points: - Used when existing memory is insufficient or excessive - Old data is preserved up to the minimum of old and new size - May return the same pointer or a new pointer - If reallocation fails, returns `NULL`

Example:

```
ptr = (int *)realloc(ptr, 10 * sizeof(int));
```

4. `free()` — Deallocation

Purpose:

Releases dynamically allocated memory back to the heap.

Syntax:

```
free(ptr);
```

Key Points: - Prevents memory leaks - After `free()`, pointer becomes dangling - Good practice:

```
free(ptr);  
ptr = NULL;
```

Important Notes (Exam Ready)

- Dynamic memory is allocated at **run time**
- Allocation occurs in **heap memory**
- `malloc()` → garbage values
- `calloc()` → zero values
- Always check allocation success:

```
if (ptr == NULL) {  
    // memory allocation failed  
}
```